Ch-08

first order Logic

* Propositional Logic (T/F)
* জটিল complex sentence এর meaning identify করা যায় না first order logic use করা হয়।
* Propositional Logic এর Drawback overcome করার জন্য first order Logic use করা হয়।
* Properties of propositional Logic
  i) Satisfiable : A atomic propositional formula is satisfiable if there is an interpretation for which is true.
  ii) Tautology : formula is valid if all possible interpretation is true.
  iii) Contradiction (unsatisfiable) : If there is no interpretation for which is true
  iv) Contingent : It means it can be neither a tautology nor a contradiction.
* Difference between Propositional Logic vs first order Logic

* First order Logic (FOL) / Predicate Logic
Another knowledge representation of AI.
which is an extended part of propositional logic.
We say that we are predicting some property of the object.

The sky is blue
object → predicate

* Facts of PL
→ work on 0 or 1
→ can be either true or false.
→ Symbolic variable are used to represent the logic and logic can also used represent variable.
→ It is comprised of objects, relation, function, logic.
→ In PL contradiction means false and tautology mean true.

* Facts of FOL
  - → Related to objects, relations and functions, properties.
  - → It has two main features
    - i) syntax ii) Semantics.
  - → Used to develop information of object.

* why we use FOL.
* Combining the best of formal and natural language
  - i) objects : people, houses, number, theories etc.
  - ii) Relations : these can be unary relations or properties such as red, round, prime.
  - iii) functions : father of, best friend etc.

Example

"One plus two equal three."

Here one, two, three → objects

functions → plus

equal → relation

"Squares neighbouring the compus are smelly"

object function object relation

Fig. 8.1

8.2.1 Models for first order logic :-

The domain of model is the set of objects or domain elements contain.

* Quantifiers

→ Universal $\forall x$ or (Ax)
→ Existential $\exists x$ (Ex)
(কিছু জিনিসকে বুঝালে)

$(\forall x)\ P(x) \rightarrow$ means P hold all values of x

$(\exists x)\ P(x) \rightarrow$ happen for some u of x

$(\forall x)$ dolphin$(x) \rightarrow$ mammal(x)

$(\exists x)$ mammal$(x) \wedge$ lays-egg(x)

Fig: 8.1

Model for FOL

Domain → Domain is the model set of objects.

Domain elements → set of objects or element of domain.

Fig: 8.2

binary relation → Relation between two or more (bination) elements.

Unary relation → 1st fig.

tuple → set of objects

* Symbols of functions →

i) Constant symbol → stands for objects
ii) Predicate symbol → stands for relation.
iii) function symbols → stands for function.

* Fig → 8.3

→ Terms
→ Atomic Sentences
→ Quantifiers
→
→

8.2.1

Model for FOL

Domain → Domain is the model set of objects.

Domain elements → Set of objects or element of domain.

fig: 8.2

binary relation → Relation between two or more (brother) elements.

Unary relation → left leg.

tuple → Set of objects

* Symbols of functions →

i) Constant Symbol → stands for objects

ii) Predicate Symbol → stands for relation.

iii) function Symbols → stands for function.

* fig → 8.3

→ Terms

→ Atomic Sentences

→ Quantifiers

→

→

* Connections between All and Exists

Universal Existential

* Translating English to FOL (Slides)

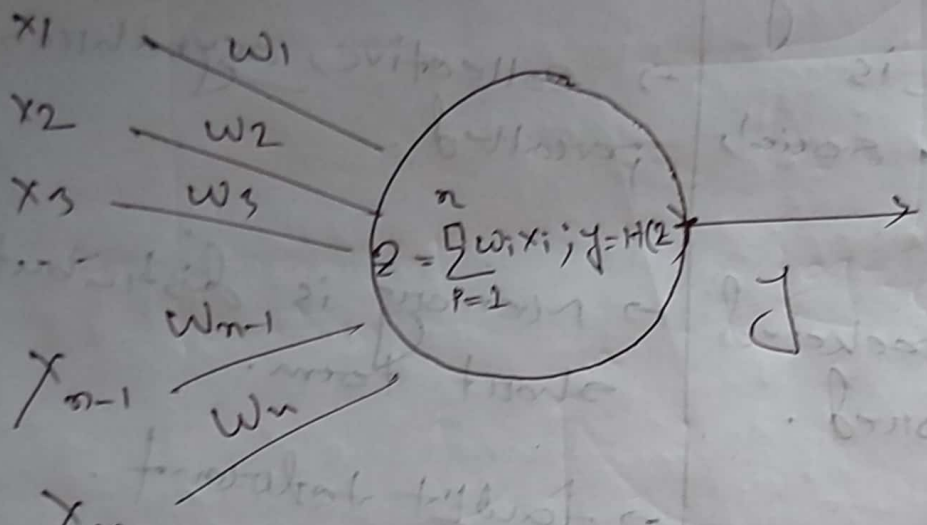
8.3 Using first-order-Logic:

Neural Network

Digital Computer	Neural Network
→ Deductive Reasoning	→ Inductive Reasoning
→ Computation is centralized, serial, synchronous.	→ collective, asynchronous, parallel.
→ Memory is packeted & literally stored.	→ Memory is distributed, short term.
→ Not fault tolerant.	→ fault tolerant.
→ Exact	→ Inexact.
→ Static connectivity	→ Dynamic connectivity

* Why neural Networks →

* Artificial Neurons' →



inputs

McCullogh-Pitts model

$$z = \sum_{i=1}^{n} x_i w_i \; ; \; y = H(z)$$

Non-linear generalization →

$$y = f(x, w)$$

y is neuron output, x is input vector of and w is graptic weights.

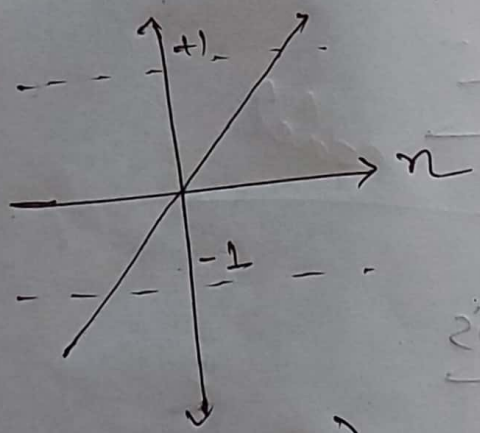
$$y = \frac{1}{1 + e^{-\omega^T x - a}}$$ Sigmoidal neuron

$$y = e^{-\frac{\|x-\omega\|^2}{2a^2}}$$ Gaussian neuron

* Activation functions →

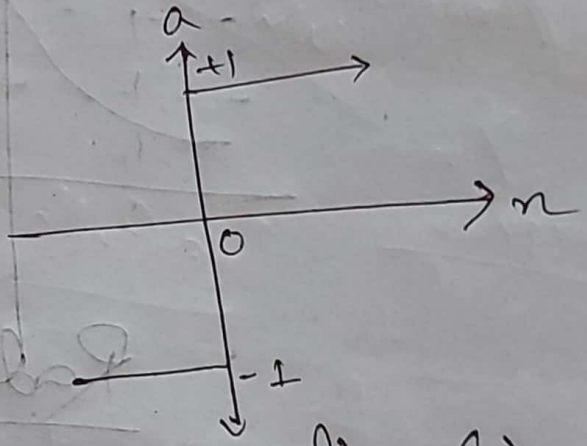
Activation function are non linear.

Linear functions are limited



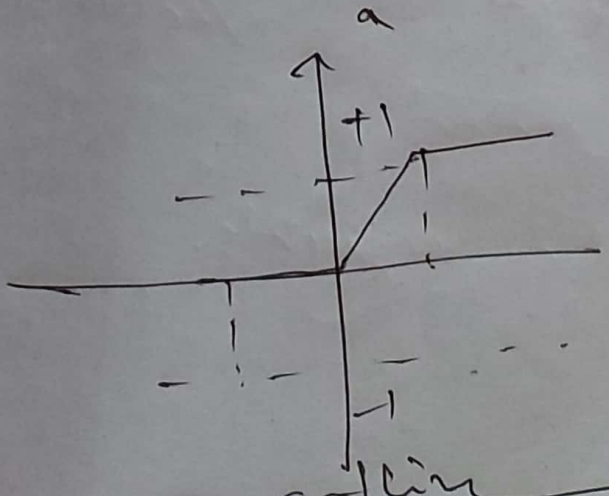
a = purelin(n)

Linear transfer function



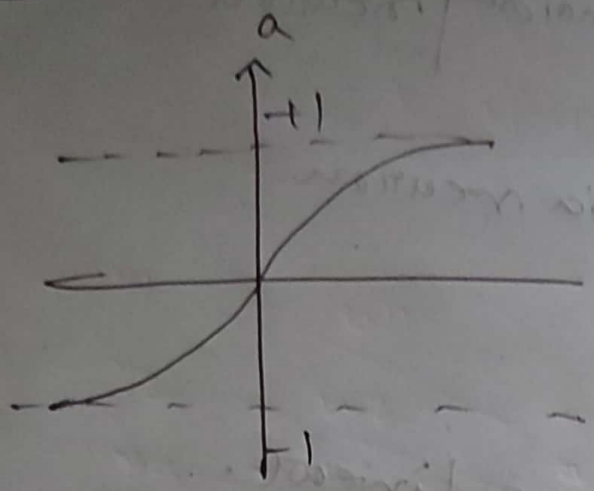
a = hardlim(n)

Symmetric Hard Limit function

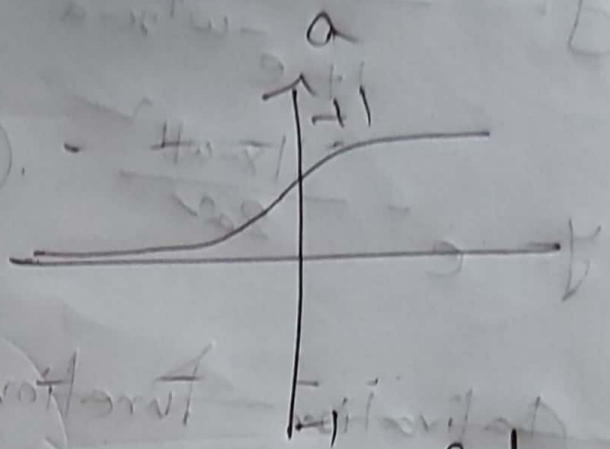


a = satlin

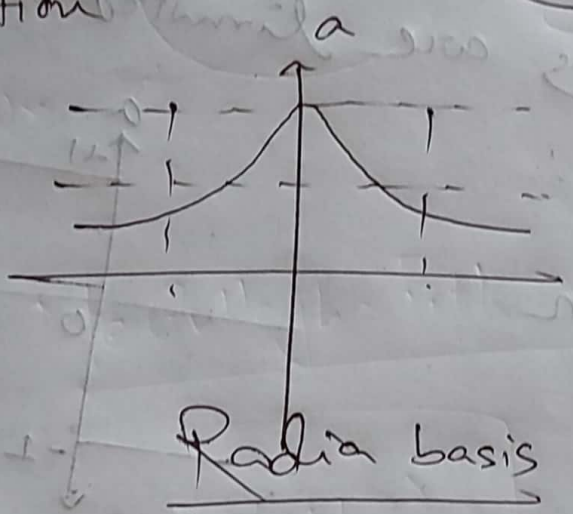
Non-Linear



Tan-Sigmoidal function



Log-Sigmoidal function



Radia basis

Book (Simon Haykin)

→ What is neural network?

→ Neural network is a method of artificial intelligence that teaches computer to process data in a way that is inspired by human brain.

* Benefits of Neural Network / machine learning / Deep learning

Useful properties and capabilities

i) Non-linearity;

ii) Input-output mapping → আউটপুট বিচার করে learn করা

iii) ~~Adaptivity~~ → নিজে adapt করতে পারে।

iv) Evidential Response

v) Contextual information.

vi) Fault tolerance

vii) VLSI Implementability.

viii) Uniformity of Analysis and design.

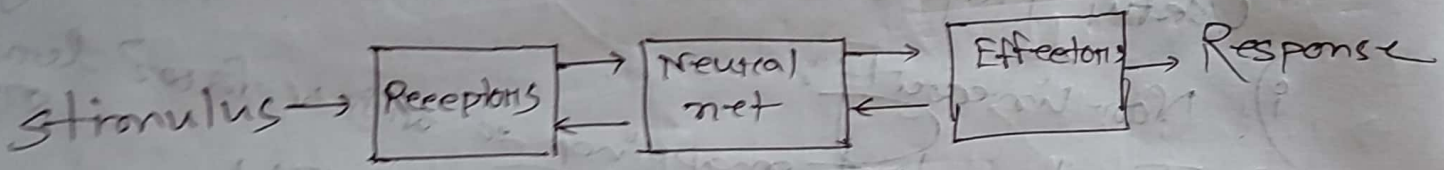
ix) Neurobiological Analogy.

Adaptivity: Highly adaptable for the environment.

VLSI → Very Large Scale integration.

i) Used for build high compeet digital IC.

* Human Brain function —



Block diagram of nervous system.

→ Structural Organization of level in the brain

Central nervous system.

↑

Interregional circuit

↑

Local circuit

↑

Neurons

↑

Dendritic trees

↑

Neural microcircuit

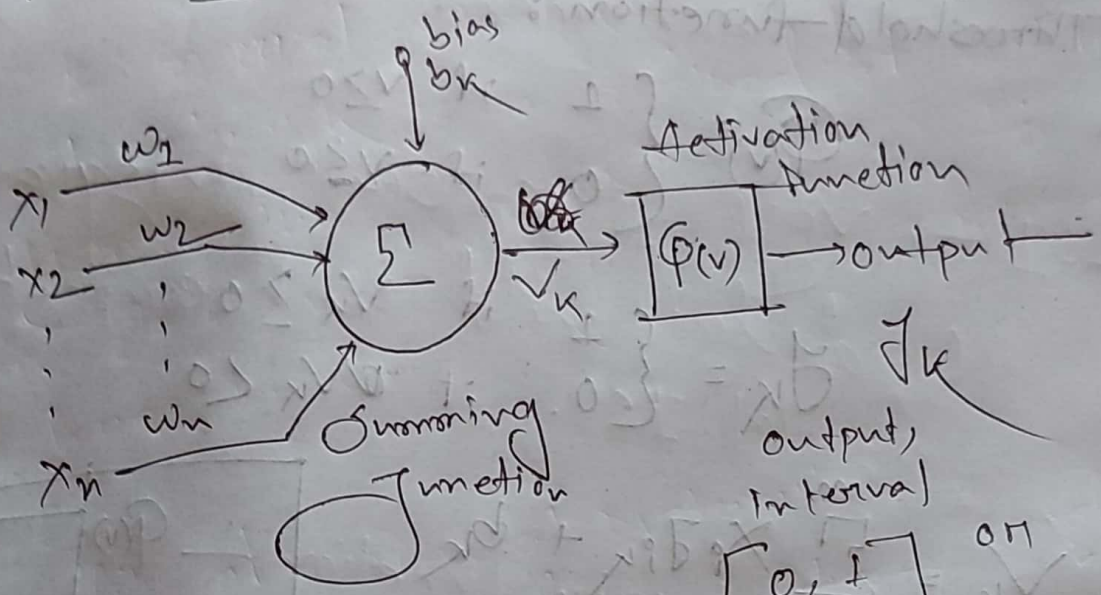
↑

Synapses

↑

Molecules

* Model of a neuron



output, interval

$[0, 1]$ or

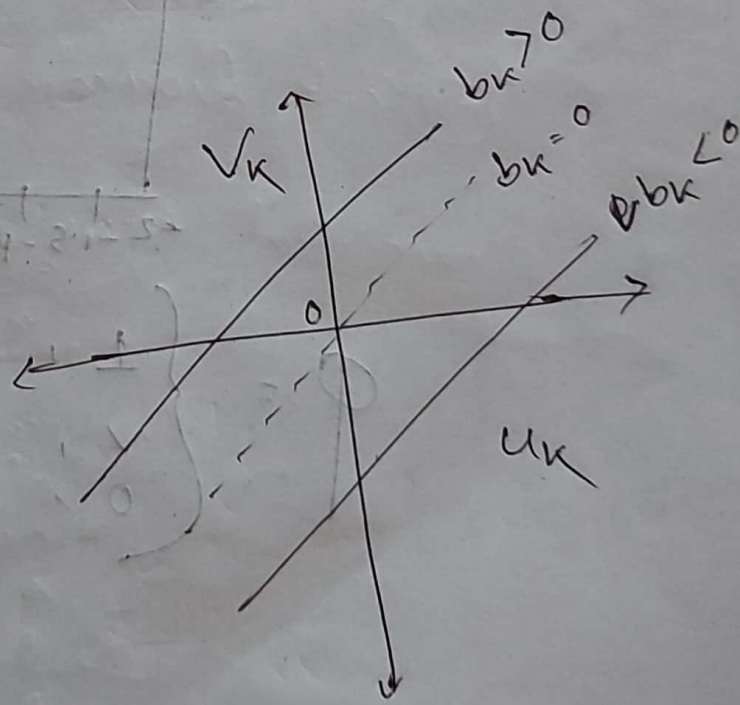
$[-1, 1]$

$$u_k = \sum_{j=1}^{m} x_i w_{jk}$$

$$y_k = \varphi(u_k + b_k)$$

$$v_k = u_k + b_k$$

$$y_k = \varphi(v_k)$$



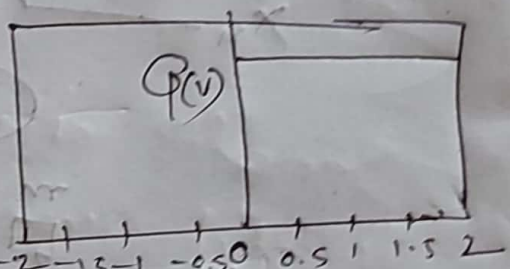
* Types of ~~Activation~~ function ($\varphi(v)$)

i) Threshold function:

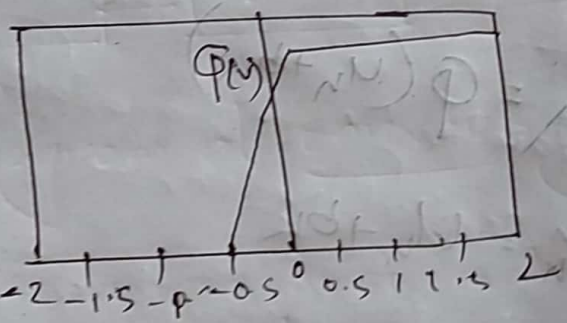
$$\varphi(v) = \begin{cases} 1 & \text{if } v \geq 0 \\ 0 & \text{if } v \leq 0 \end{cases}$$

$$y_k = \begin{cases} 1 & \text{if } v_k \geq 0 \\ 0 & \text{if } v_k < 0 \end{cases}$$

$$v_k = \sum_{j=1}^{m} x_i w_{ik} + b_k$$

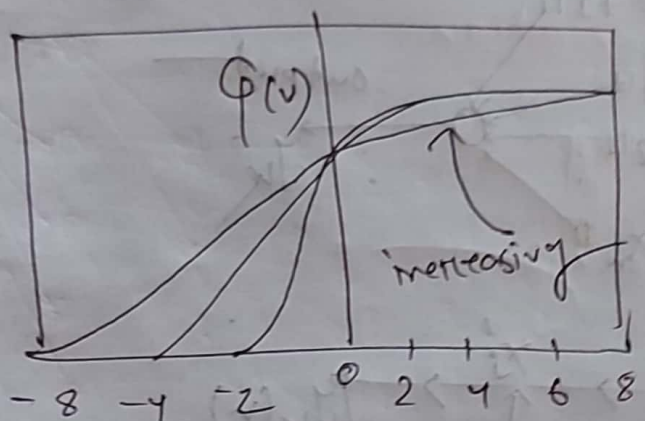


(ii) Pieee-wois Linear function:



$$\varphi = \begin{cases} 1, & v > \frac{1}{2} \\ v, & +\frac{1}{2} > v > -\frac{1}{2} \\ 0, & v \leq \frac{1}{2} \end{cases}$$

3. Sigmoid function:



$$\varphi(v) = \frac{1}{1+\exp(-av)}$$

$$\varphi(v) = \tanh(v)$$

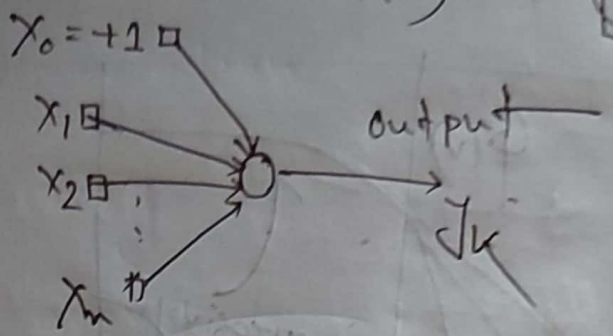
* Stochastic model:-

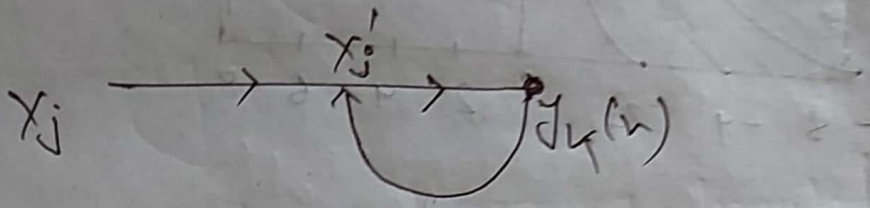
$$x = \begin{cases} +1 & \text{with probability } P(v) \\ -1 & \text{" " } 1-P(v) \end{cases}$$

$$P(v) = \frac{1}{1+\exp(-v/T)}$$

* Feedback:

$$y_k(n) = A\left[x_j'(n)\right]$$





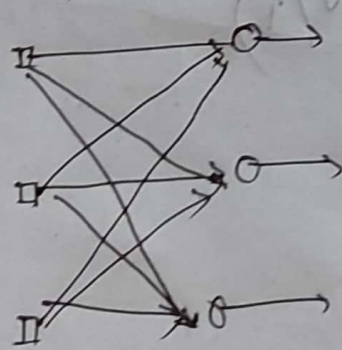
Single flow feedback Loop

নিজে নিজেই Learn করে। output যখন আবার input এ যায় তখন সেটি Adaptive Learning হিসেবে কাজ করে।

* Network Architecture

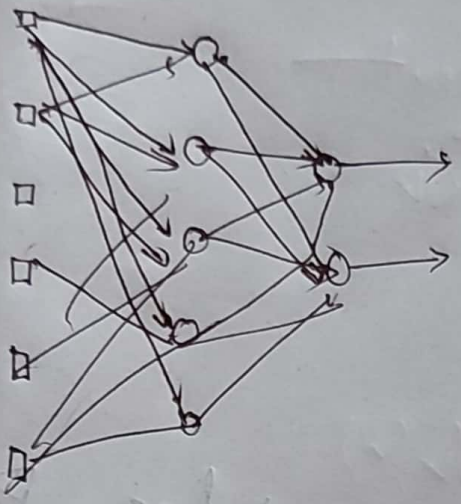
i) Single Layer forward network
ii) Multi Layer " "

i)



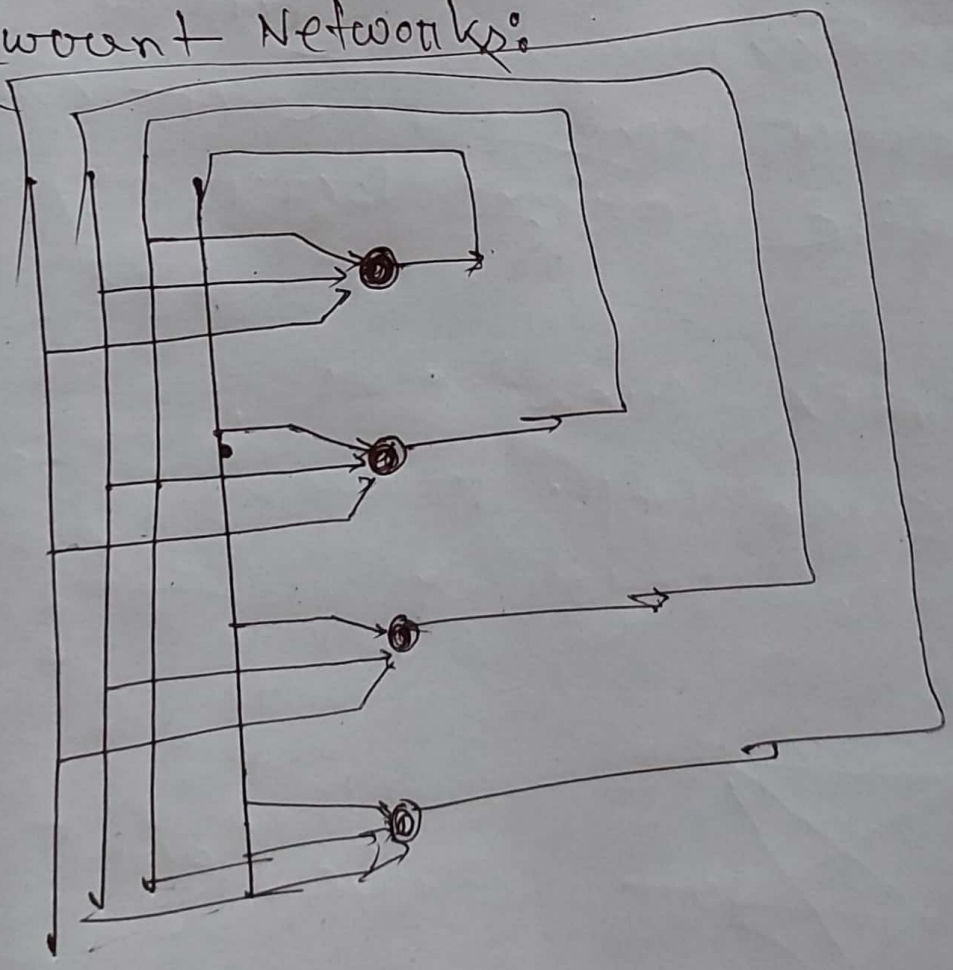
single Layer.

(ii) One or more hidden layer multi layer



Connection → one to all

(iii) Rewrant Networks:

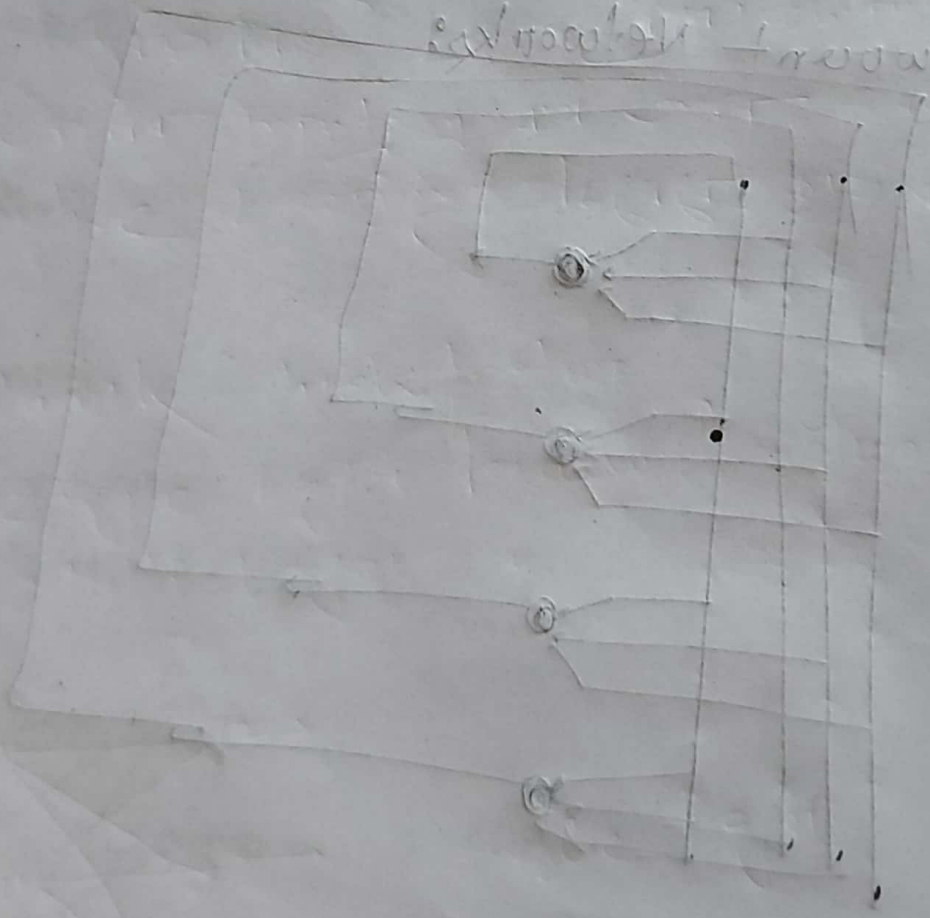


1.8 AI and Newral Network

1) Representation
2) Learning
3) Reasoning

* Learning →

Environment → Learning element → knowledge base → Performance element



Artificial Intelligence

* Searching Algorithm →

i) Uninformed (Blind Search)

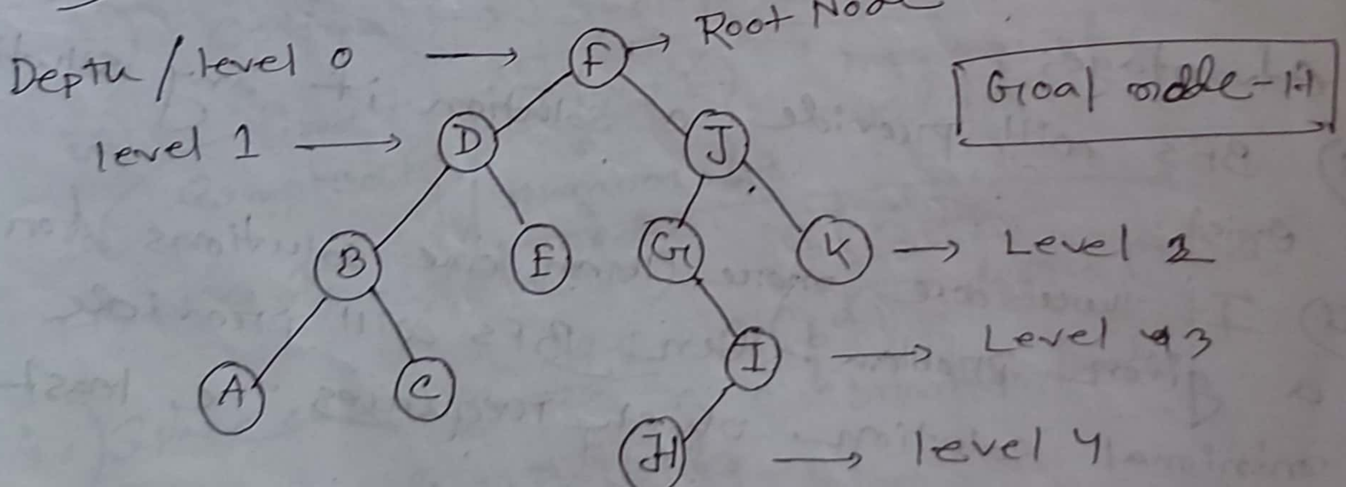
ii) Informed (Heuristic Search / direct search) → knowledge

i) Uninformed (কোনো info দেওয়া থাকে না।)

1) Breath first search
2) Depth-first Search

ii) Informed
1) Best first search (Greedy)

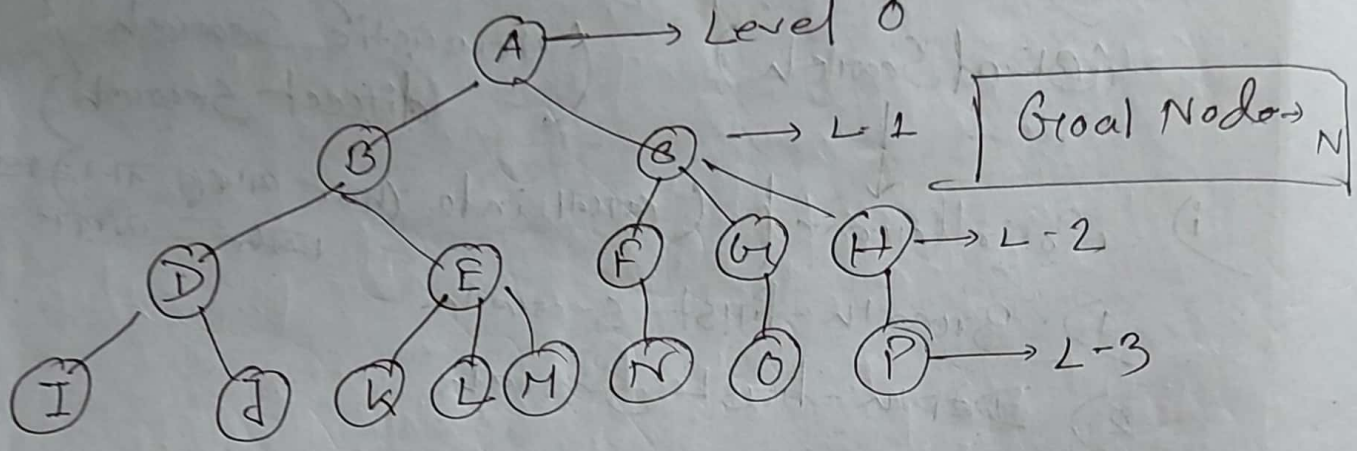
1) Breath-first Search: (FIFO Queue)



Left to Righ BFS →

F, D, J, B, E, G, K, A, C, I, H

BFS → root node থেকে শুরু করতে হবে।
তারপর একটি লেভেল শেষ হওয়ার পর
অন্য লেভেলে যাবে। BFS L → R করতে হবে।



BFS → A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P.

[Level (BFS) ~~order~~ Traversal order. (L→R)]

[N পর্যন্ত লেখলে হবে।]

→ Advantages :

i) BFS will provide a solution if any solution exist.

ii) If there are more than one solutions for a given problem, then BFS will provide minimal solution, which requires the least number of steps.

iii) BFS will find the shortest path between the starting point and any other reachable node.

Disadvantages:

i) It requires lots of memory since each level of the tree must be saved into memory to expand the next level.

ii) BFS needs lots of time if the solution is far away from the root node

BFS → BFS algorithm starts searching from the root node of the tree and expands all the successor node at the current level before moving to node of next level. use queue data structure.

Properties of BFS →

i) Complete → complete result for 1

ii) Optimal → shortest path.

iii) Time Complexity → $O(V+E)$ → number of edges

number of nodes → $1+b+b^2+b^3+\cdots+b^d = O(b^d)$

↓ in D.S.

In AI $O(b^d)$ → depth (level)

↓ branch factor.

iv) Space

b = a node to children

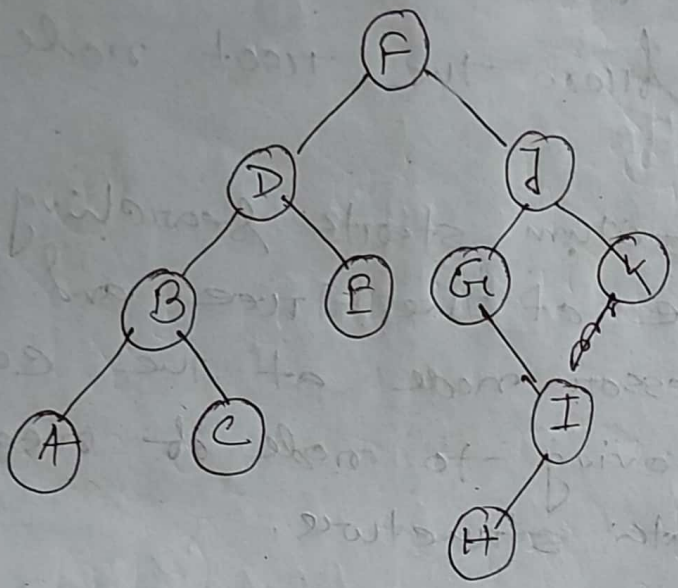
* DFS (Depth first Search) [Intent: depth Node খুঁজে বার করা]

3 popular strategy →

i) Pre-order ii) In-order iii) Post order.

↓

root → L to R L → Root → R L - R - Root



i) Pre-order → F, D, B, A, C, E, J, G, I, H, K

ii) In-order →

iii) Post-order →

→ Uninformed
→ Stack (LIFO)
→ Depest Node
→ Incomplete
→ Non-optimal
→ Time Complexity → $O(b^d)$

d → depth

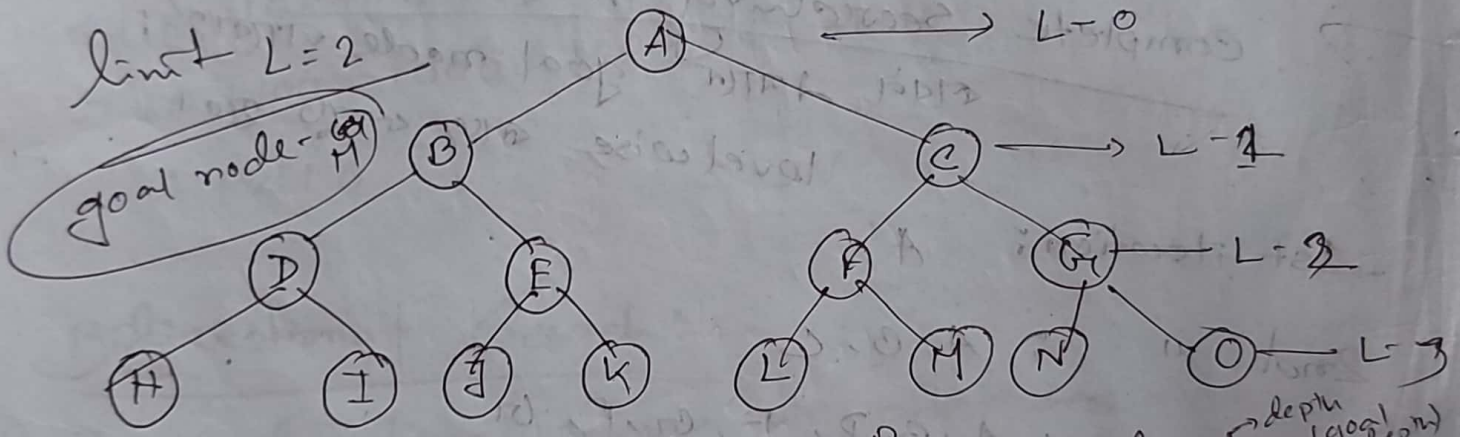
b → branding factor (কতগুলো children possible এর সংখ্যা)

* Depth limited Search : (DLS)

Limitation of DFS (Incomplete)

DFS + Limit → DLS

DFS → infinit path- node → যেতে থাকে তবে জন্য goal node → যেতে problem হয়। Infinite DFS → limit করে দেওয়ার জন্য DLS use করা হয়।



Root - L → R

A, B, D, E, C, F, G

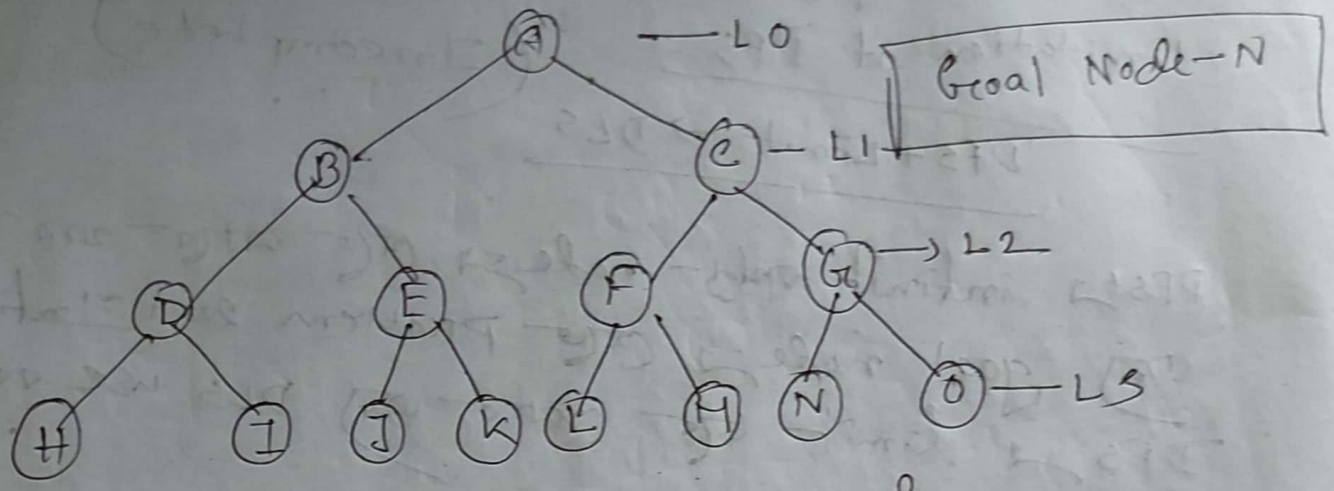
$l < d$ → depth (goal depth) Incomplete

$l > d$ → optimal (level)

এখানে, M হচ্ছে limit level এর বাইরে। DLS এ problem হচ্ছে Level এর বাইরে যদি goal থাকে তাহলে তাহলে তা ধরা যায় না।

এই problem solve করার জন্য iterative deeping Search use করা হয়।

* Iterative Deepening Search (IDS):



Complete searching method

কারণ এখানে goal node পাবেই।

level wise করে করতে হয়।

1st iteration: A

2nd " : A, B, C

3rd " : A, B, D, E, G, F, G

4th " : A, B, D, H, I, E, J, K, C, F, L, M, G, N, O

Complexity:

$N(IDS) \rightarrow (d)b + (d-1)b^2 + \cdots + 1\,b^d$

Time Complexity: $O(b^d)$

order of

Here, $b=10$, $d=5$

$$N(IDS) = (d)b + (d-1)b^2 + \dots + d(1)b^d$$

$$= (5)10 + (5-1)10^2 + (5-2)10^3 + (5-3)10^4 + (5-4)10^5$$

$$= 123,450$$

$$N(BFS) = b + b^2 + b^3 + \dots + b^d$$

$$= (10)^1 + (10)^2 + (10)^3 + (10)^4 + 10^5$$

$$\Rightarrow 10 + 100 + 1000 + 10000 + 100000$$

$$= 111,110$$

* Bidirectional Search:

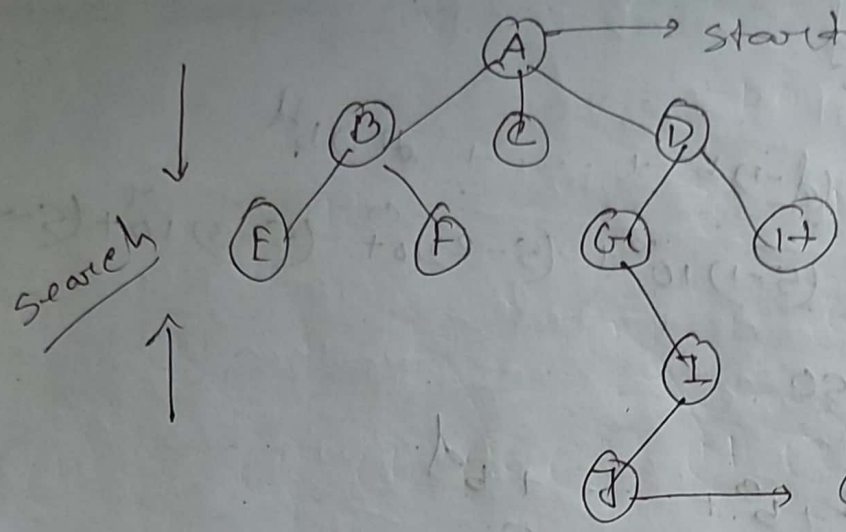
i) In Bi-directional search we start searching from both directions simultaneously.

ii) forward search from start node.

iii) Backward search from goal node.

iv) We can use any searching method such as BFS / DFS

*



(time compleatly কম) বারণ- both side থেকে search করা = তার- খুব fast goal node পাওয়া যায়। দুই search-এ যখন একই node মিলে যাবে তখনি stop হয়ে- যাবে।

start node (BFS)

→ A, B, C, D

Goal node (DFS)

→ J, I, G, D

→ Time → $2(b^{d/2}) \rightarrow (b^{d/2} + b^{d/2})$

→ Complete in breath first Search

→ Not a in Depth first Search

* Greedy Search / Greed best first Search:

→ Complete in finite space
→ incomplete in infinite space

Time complexity → $O(b^m)$
Space → $O(b^m)$

m = maximum depth of search space.

* A* search: (avoid expensive path)

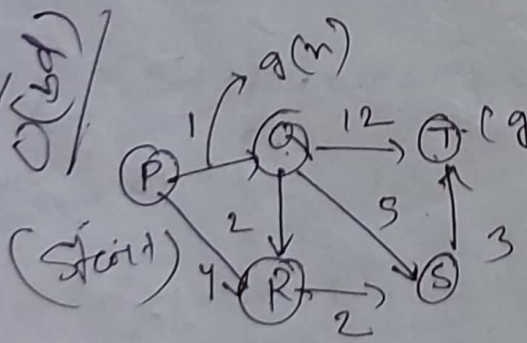
Evaluation function, $f(n) = g(n) + h(n)$

$g(n)$ → the cost to reach the node

$h(n)$ → estimated cost to get from the node to the goal.
(heuristic value) (total cost)

$f(n)$ → estimated cost of path through n to goal.

Complete and optimal / $O(b^d)$



(start to goal) এর h(n)

n	h(n)
P	7
Q	6
R	2
S	1
T	0

P → P 0+7 = 0
P → R → 4+2 = 6
✗ R → Q → 1+6 = 7 (used)
P → R → S → 4+2+1 = 7 ✗
P → Q → R → 1+2+2 = 5 ✗
P → Q → S → 1+5+1 = 7 ✗
P → Q → T → 1+12+0 = 13 ✗ (সব চেয়ে বড়)

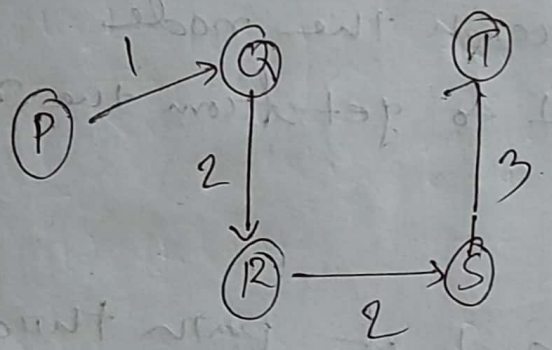
$R \to Q - R \to S = 1+2+2+1 = 6$ ✗

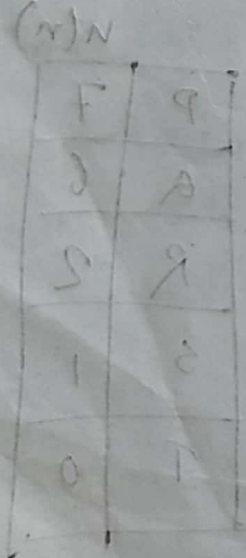
$P \to Q \to R \to S \to T - 1+2+2+3+0 = 18$

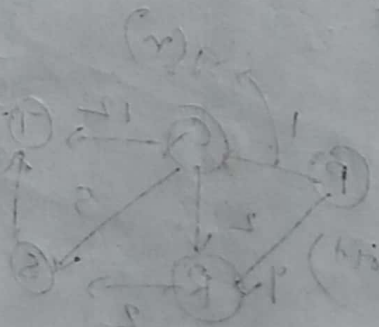
$P \to R \to S \to T \to 4+2+3+0 = 9$ ✗ (wrong)

$P \to Q \to S \to T \to 1+5+3+0 = 9$

→ for cost path
A* search







A* Search:

Informed Search:

Greedy Best first Search:

Heuristic function h(n):

h(n) Estimate cost of the cheapest path from node n to goal node.

How to make heuristic function.

7	2	4
5	-	6
8	3	1

Start state

-	1	2
3	4	5
6	7	8

Goal state

h_1 = total number → $h_1 = 8$

h_2 = 3+1+2+2+2+3+3+2 = 18

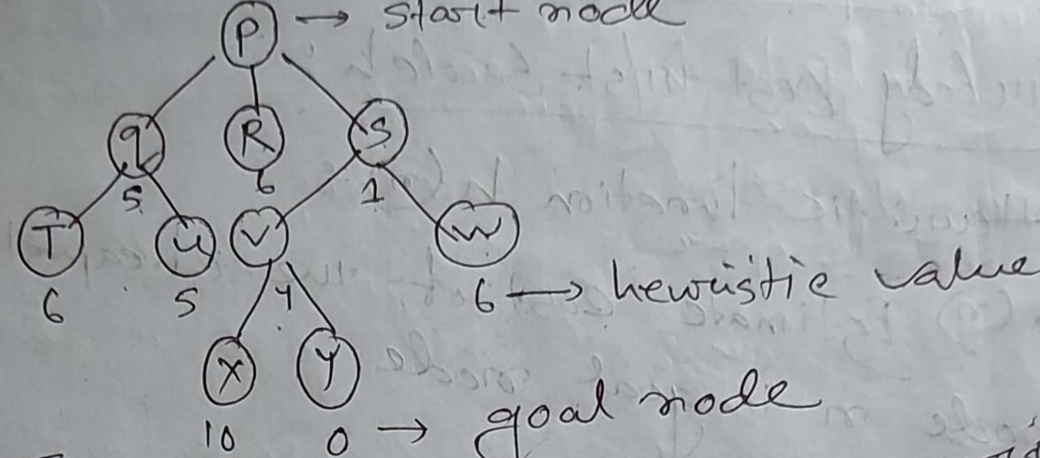
h_2 ↓ move number

$h(n) = h_1 + h_2 = 18 + 8 = 26$

max → 10 বার move করলে goal state পাওয়া যাবে কম সংখ্যক ও হতে পারে

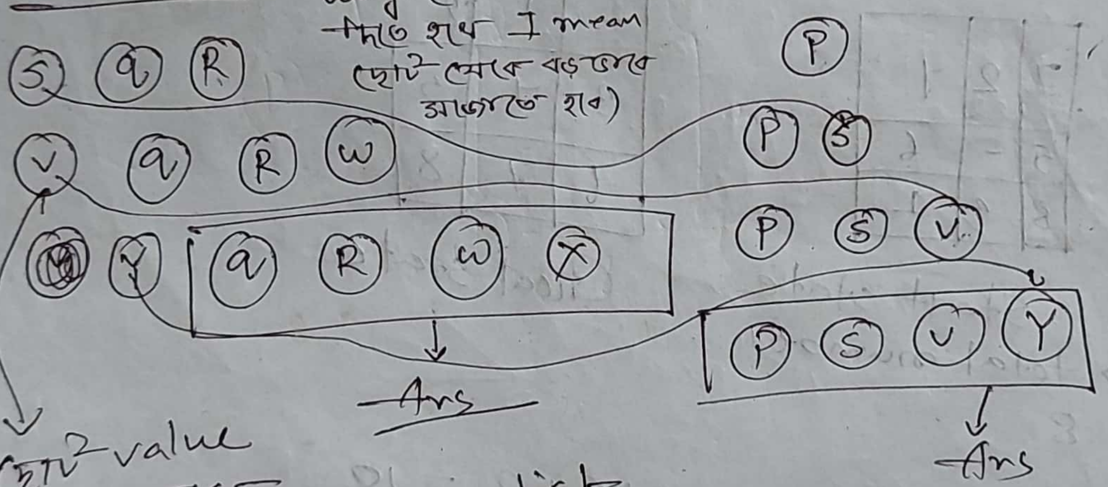
* Best first Search:

Combination of BFS and DFS →



(যত পথ বের হবে তোমার)

Openlist (heuristic way-তে value কমে হবে I mean ছোট থেকে বড় ভাবে সাজাতে হবে) — closed list (যে node থেকে path বের হবে)



ছোট value গুলোকে node দিবে open list and closed list এ বসাতে হবে।

[space and time complexity → $O(b^m)$

m = maximum depth of Search tree.]

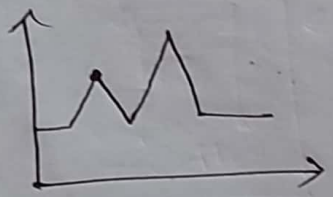
[greed best first Search]

Local Search Algorithm

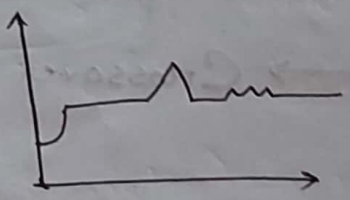
i) Hill climbing ii) Simulated annealing
iii) Local Beam Search iv) Genetic
Search Algorithm.

i) Hill climbing Drawback →

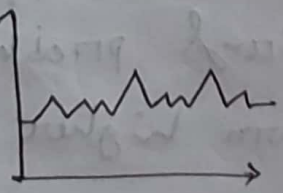
→ Local Maxima: Algorithm can't go higher, but not at a satisfactory solution.



→ Plateau: Area where evaluation function is flat.



→ Ridges: Search may oscillate slowly.



Hill climbing a is also called greedy local search.

* Hill climbing Vs Simulated Annealing

→ Hill climbing → Local maxima তে এসে algorithm stop করে দেয়।

→ Simulated annealing best চূড়া না পাওয়া পর্যন্ত Algo. চলতেই থাকে। best চূড়া না পেলে previous state → ফিরে best path search করতে থাকে।

Hill climbing এর problem overcome করার জন্য simulated annealing use করা হয়।

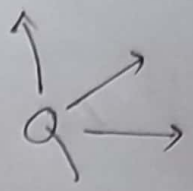
Genetic Algorithm

selection → Crossover → mutation

(according fitness)

fitness function: fitness function evaluates how close a given solution is to the optimal solution of the desired problem. A fitness function should return higher values for better states.

Attacking pair →



$q_1 = 5$

$q_2 = 5$

$q_3 = 3$

$q_4 = 6$

$q_5 = 7$

$q_6 = 4$

$q_7 = 5$

$q_8 = 5$

$40/2 = 20$

$q_1 = 3$

$q_2 = 4$

$q_3 = 3$

$q_4 = 2$

$q_5 = 1$

$q_6 = 1$

$q_7 = 3$

$q_8 = 5$

$22/2$

$= 11$

* n সংখ্যক queen দ্বারা কতগুলো non-attacking number তৈরি হবে —

$$\frac{n \times (n-1)}{2}$$

Ch-06

Constraint satisfaction Problems (CSP)

Example: Map Coloring

CSP → A problem is solved when each variable has a value that satisfies all the constraints on that variable is called CSP.

Defining CSP

CSP consist of three components

X → set of variable $\{X_1, \ldots, X_n\}$

D → set of Domain $(D_1, \ldots, D_n)$ one for each variable (values)

C → set of constraints that specify allowable combination of values.

{ variable, value }

Scope, Relation

* Region colour (value) →

D_i = Domain = {red, green, blue}

X = Variable = {WA, NT, SA, Qld, NSW, V}

Constraints, C = {SA ≠ WA, SA ≠ NT. . . . }

for example —

The allowable combinations for WA and NT are the pair { (red, green), (red, blue), (green, red), (green, blue), (blue, green), (blue, red) }

WA ≠ NT, provided the constraints satisfaction algorithm has some way to evaluate such expressions.

There are many solution. One possible is shown →

{ WA = red, NT = green, Q = red, NSW = green,
V = red, SA = blue, T = Red / Green / Blue }

without taking adv. of contraints propagation, total combination = 3^5 = 243

with taking contraints combination of color = 2^5 = 32

Reduction 87%

* Contraints → ($S \neq$ Blue)

i) Unary → restrict value of single variable

ii) Binary → Two variable ($S \neq N$)

iii) Global → All diff

iv) Linear → each variable form linear

v) Non-linear → non linear variable

* Cryptarithmetic Problem →

$$\begin{array}{r} c_3\ c_2\ c_1 \\ T\ W\ O \\ +\ T\ W\ O \\ \hline F\ O\ U\ R \end{array}$$

X (variable) = {T, W, O, F, U, R}

D (Domain) = {0, ..., 9}

Constraint

$O + O = R + 10 * c_1$

$c_1 + W + W = U + 10 * c_2$ $c_1 \{0, 1\}$

$c_2 + T + T = O + 10 * c_3$

$c_3 = F$

Constraint = i) digit shouldn't Repeat for another alphabet

ii) sum ≤ 19 ↓ carry

iii) MSB কখনো 0 হবে না,

iv) Least possible value নিতে হবে

F = 1 [কারণ F=0 হলে 3 টি value হয়ে যাবে but এখানে 4 টি value নিয়ে কাজ করতে হবে]

T	~~6~~ 7
W	3
O	~~2~~ 4
F	1
U	6
R	~~4~~ 8

T = {0, 1} হওয়া যাবে না [কারণ — 0 হলে 2 টি value হয়ে যাবে (TWO) — এ আবার 1 হবে না কারণ F = 1 assign হয়ে গেছে]

Possible value is = {2, 3 . . . 9}

(2–6) পর্যন্ত হবে না,

6 + 6 = 12 carry, T = ~~6~~ 7 হলে

O = 4 হবে

O + O = ~~12~~ 8

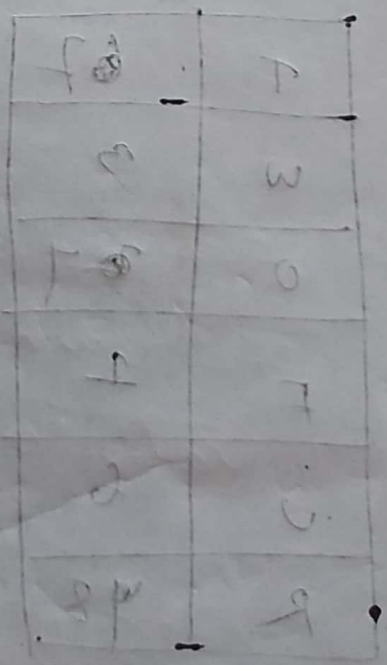
4 + 4 = 8

* Sudoku example

Constraints → i) Row wise (1-9) ஒரு முறை
ii) Columnwise (1-9) ஒரு முறை
iii) Per square (1-9) ஒரு முறை

Variable → Each of the 81 squares.

Domain → each number (1-9)



Ch-07 (Book-2)

Reasoning and Uncertainty

Uncertainty / Probability

Probability / Uncertainty → came from random experiment

Random Variable → ~~Same~~ value from random experiment.

why uncertainty → i) Uncertain input
ii) Uncertain knowledge.

Prior Probability → Prior probability is the probality of an event base on established knowledge, before empirical data is collected.

✓ P(A), P(B), P(H), P(T) ... .

(Omega) Ω = Random Sample Space → {T, H}

Ω = Certain event

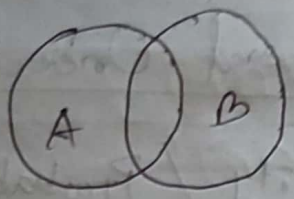
∅ = impossible event

IId → Identical Independent.

Prior probability → P(A), P(B), P(C) সব value (কারো উপর depend করে না)

Joint probability → P(A, B).

(Calculate of two events occuring together and at the same point)



$P(A \cap B)$ / $P(A, B)$

(joint probability)

Ex-72

Joint $P(5, 6) = \frac{2}{6} = \frac{1}{3}$

Set Notation →

Ω → certain value

$\emptyset$ → f

$P(A \cup B) = P(A) + P(B)$

$P(A) + P(\neg A) = 1$

$1 + 0 = 1$

→ $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

→ $A \subseteq B = P(A) \leq P(B)$

→ $A_1, A_2, \ldots, A_n$ $\sum_{i=1}^{n} P(A_i) = 1$

$$\sum_{i=1}^{n} P(A_i) = 1$$

Conditional Probability:

$$P(G|S) = \frac{P(A \cap B) \;/\; P(A,B)}{P(B)}$$

Probability (G given S)

$$P(A|B) = \frac{P(A \cap G \cap S) \;/\; P(G,S)}{P(S)}$$

Probability of A

$P(A \cap B) = P(A) \cdot P(B)$ [independent]

P(A) and P(B) when independent.

$$P(G|S) = \frac{P(G \cap S)}{P(S)} = \frac{5}{30} = \frac{1}{6} \approx 0.17$$

Probability G given S, P(G|S)

$$P(G) = \frac{P(G \cap S)}{P(S)} = \frac{P(G) \cdot P(S)}{P(S)}$$

$$= P(G)$$

This event called, independent

Chain Rule:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$

$$P(A \wedge B) = P(A|B) \cdot P(B)$$

~~for~~ n variable

$$P(X_1, X_2, \ldots X_n) = P(X_n | X_1, X_2 \ldots X_{n-1}) \; P(X_1, X_2 \ldots X_{n-1})$$

$$= P(X_n | X_1, X_2, \ldots X_{n-1}) \; P(X_{n-1} | X_1, X_2 \ldots X_{n-2}) \; P(X_1, X_2 \ldots X_{n-2})$$

$$= P(X_n | X_1, \ldots X_{n-1}) \; P(X_{n-1} | X_1 \ldots X_{n-2}) \; \cancel{P(X_1 \ldots X_{n-2})}$$

$$P(X_{n-2} | X_1 \ldots X_{n-3}) \; P(X_1 \ldots X_{n-3})$$

$$\Rightarrow \prod_{i=1}^{n} P(X_n | X_1 \ldots X_{i-1})$$

$$P(X_1, X_2, \ldots X_n) = P(X_n | X_2 \ldots X_{n-1}) (X_1 \ldots X_{n-1})$$

$$= P(X_n | X_1 \ldots X_{n-1}) P(X_{n-1} | X_1 \ldots X_{n-2})$$

$$P(X_1 \ldots X_{n-2})$$

$$= P(X_n | X_1 \ldots X_{n-1}) P(X_{n-1} | X_1 \ldots X_{n-2})$$

$$P(X_{n-2} | X_1 \ldots X_{n-3}) P(X_1 \ldots X_{n-3})$$

$$= \prod_{i=1}^{n} P(X_n | X_1 \ldots X_{i-1})$$

* Bayes theorem: → Based on conditional probability

$$P(A|B) = \frac{P(A \wedge B)}{P(B)} \quad \text{— (1)}$$

$$P(B|A) = \frac{P(B \wedge A)}{P(A)} \quad \text{— (2)}$$

Marginalization:

$$P(A) = \cancel{P(A) \wedge P(B)} \quad P(A \wedge B) \vee P(A \wedge \neg B)$$

$$= P(A \wedge B) + P(A \wedge \neg B)$$

$$P(A \wedge B) = P(B \wedge A)$$

$$\Rightarrow P(A|B)\,P(B) = P(B|A)\,P(A) \rightarrow \text{posterior}$$

$$\Rightarrow P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$$

↓ Likelihood — ↳ Bayes Rule

$$P(App|\,Leuko) = \frac{P(Leuko|\,App)\;P(App)}{P(Leuko)}$$

$$= \frac{P(Leuko \wedge App)\;P(App)}{P(App)\;P(Leuko)}$$

$$P(Pos) = P(Pos \wedge C) + P(Pos \wedge \neg C)$$

$$\rightarrow P(Pos)\cdot P(C) + P(Pos)\cdot P(\neg C)$$

ch-13 (Russel)

$0 \leq P(\omega) \leq 1$ for all ω $\quad \sum_{\omega \in \Omega} P(\omega) = 1$

Probability with a proposition:

$$P(\text{total} = 11) = P((5,6)) + P((6,5))$$

$$= \frac{1}{36} + \frac{1}{36}$$

$$P(a|b) = \frac{P(a \wedge b)}{P(b)}$$

$$P(a \wedge b) = P(a|b)\, P(b)$$

Product Rule

Example: $T = \{\text{Cavity}, \text{Toothache}, \text{Catch}\}$

$\text{Dice} = \{1, 2, 3, 4, 5, 6\}$

Age :=

weather :=

$$P(Y) = \sum_{z \in Z} P(Y, z)$$

$P(\text{cavity}) = 0.108 + 0.012 + 0.072 + 0.008 = 0.2$

$P(\neg\text{cavity}) = 0.016 + 0.064 + 0.144 + 0.576$

$P(\text{toothache}) = 0.108 + 0.012 + 0.016 + 0.064 = 0.2$

$P(\neg\text{toothache}) = 0.072 + 0.008 + 0.144 + 0.576$

$P(\text{cavity} \vee \text{toothache}) = P(\text{cavity}) + P(\text{toothache}) - P(\text{cavity} \wedge \text{toothache})$

$= 0.2 + 0.2 - (0.108 + 0.012)$

Bayes Theorem:

$$P(C|T) = \frac{P(C \wedge T)}{P(T)} = \frac{0.018 + 0.012}{0.28}$$

$$P(C|T \vee \text{catch}) = \frac{P(C \wedge (T \vee \text{catch}))}{P(T \vee \text{catch})}$$

$$= \frac{P(C \wedge T) \wedge P(C \wedge \text{catch})}{P(C \wedge T) \vee P(C \wedge \text{catch})} \quad \frac{0.108 + 0.012}{0.28 + 0.28}$$

$$P(a|b) = \frac{P(a \wedge b)}{P(b)}$$

$$P(a|b)\ P(b) = P(a \wedge b)$$

$$P(b|a)\ P(a) = P(b \wedge a)$$

$$P(a|b) = \frac{P(b|a)\ P(a)}{P(b)} \quad \text{[Bayes Rule]}$$

$$P(a \wedge b) = P(a|b)\ P(b) \quad \text{[Product ru}$$

Masud Sir
Artificial Intelligence

10.08.22 (Missed class)

Problem Definition and formulation:

= ~~Agent~~ → Receiving information through sensors from environment.

= Goal based ~~Agent~~.

* A problem can be defined formally by 5 components:

i) the initial state of the agent.
ii) The possible action.
iii) The transition model.
iv) Goal test.
v) The path cost function.

Optimal Solution →

frontier → set of leaf nodes.
leaf nodes, Loopy path → infinite, unsolvable, explored set.

* Uniform cost search:

for many step-cost function. UCS expands the node with least path cost. To implement this, the frontier will be stored in a priority queue

How many step doesn't care:

less cost

Total Solution cost = c^*

each step cost at least = ϵ

steps = $1 + c^*/\epsilon$

time complexity = $O(b^{[1+c^*/\epsilon]})$

space " = $O(b^{[1+c^*/\epsilon]})$

* DFS এর problem হলো কোন address নিবে।

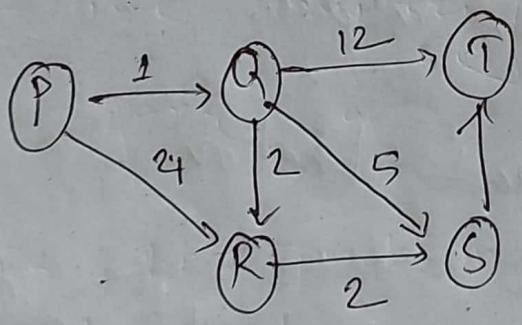
→ limitations

* Informed Search →

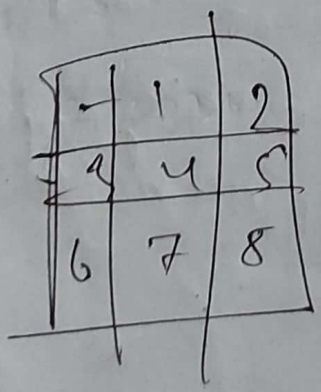
Informed search algorithm have information on the goal state which helps in more efficient searching.

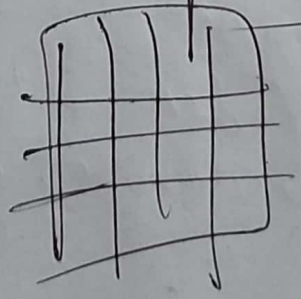
$f(n) = h(n) + g(n)$

- $f(n)$ → estimate the cost from n to goal
- $h(n)$ → estimate the cost to get the goal
- $g(n)$ → the cost to reach the goal



P = 7
Q = 6
R = 2
S = 1
T = 0

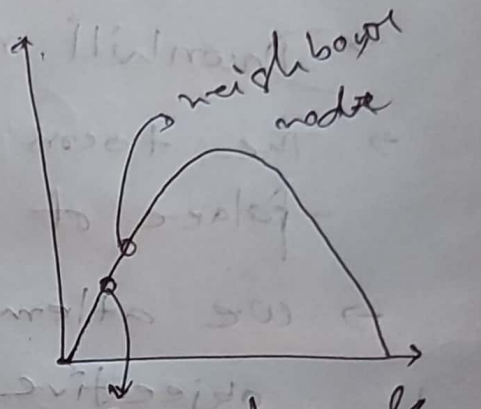




* Hill climbing / Local Search / Local beam Search

Algorithm:

1. local variable:
   Current, a node
   Neighbor, a node.



2. Current node ← Initial node
3. Evaluate the initial state and if it is goal state then return and stop.
4. Loop until a new solution found or a new node found.
5. Select and apply new node.
6. Evaluate the new node state and if it is goal then return and stop.
7. If it is better then the current then make it new current state.
8. If it is not better then the current state then go to step 4.

* Simulated annealing Search:

→ Variation of Hill climbing method.

→ Downhill move may be made.

→ The term objective function is used in place of heuristic function.

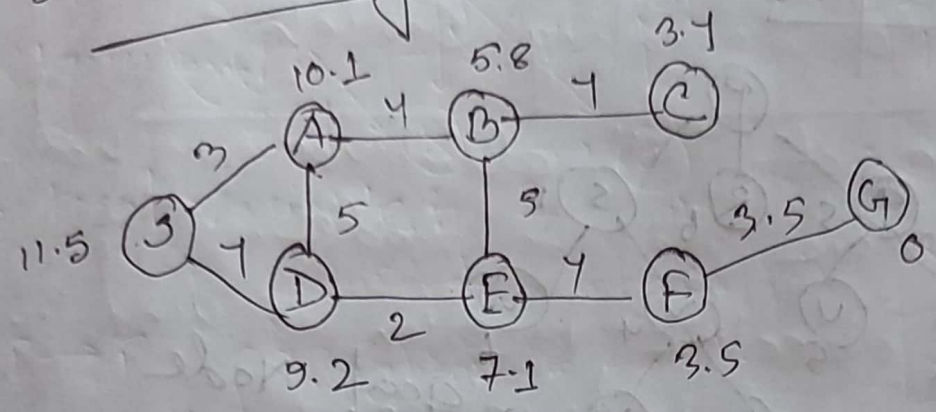
→ we attempt to minimize the objective function.

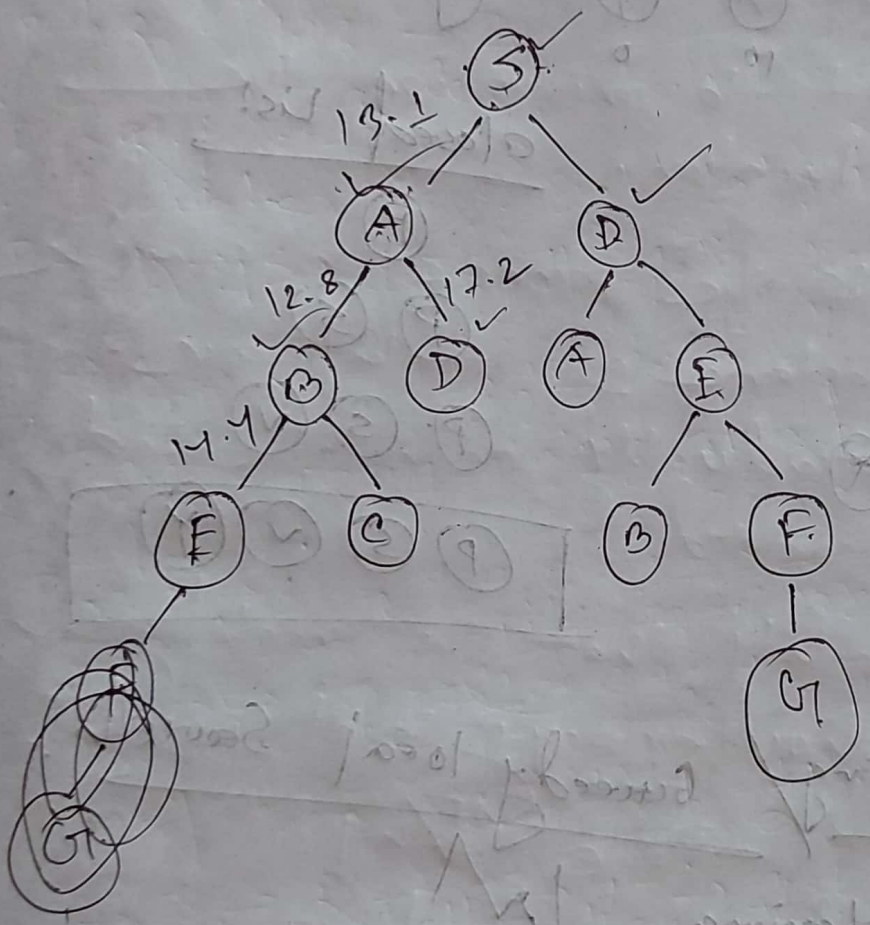
→ Annealing is the process in which physical substance as metal are melted and then gradually cooled until solid state is reached.

The probability that the metal jump to a higher energy level is given by, $P = e^{-\Delta E/kT}$

Probability of making bad move = $P' = e^{-\Delta E/T}$

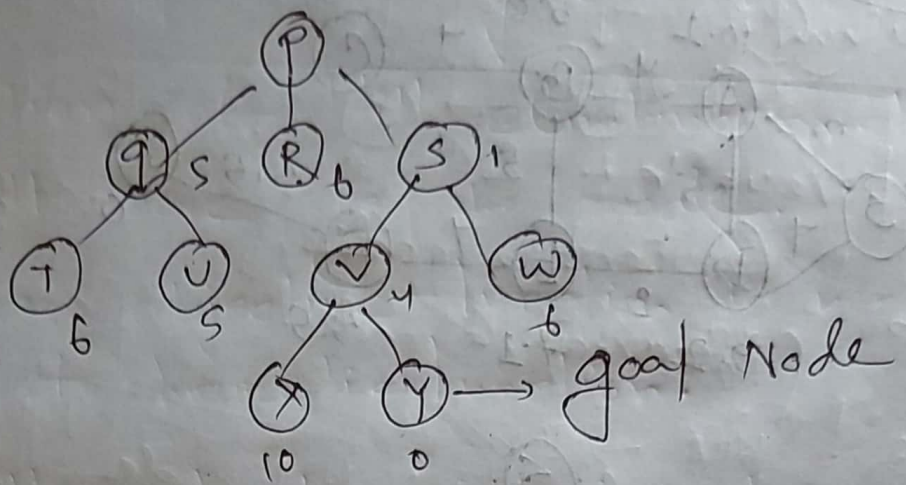
A* search Algorithm :





S → D → E → F → G

* Best first Search



Open list

S Q R

V Q R W

Y Q R W X

closed List

P

P S

P S V

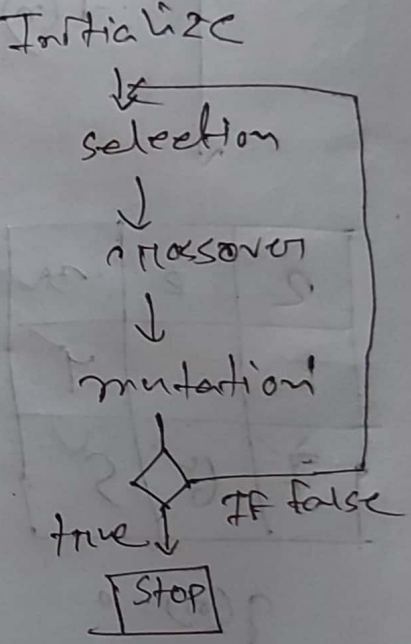
P S V Y

* Hill climbing / Greedy local Search -

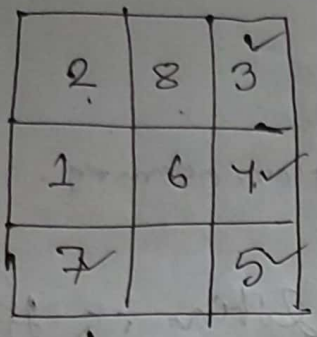
1. Local Maxima
2. Plateau
3. Ridge.

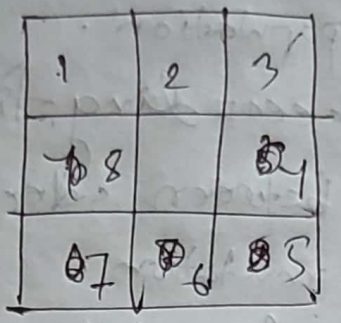
* Genetic Algorithm :

i) Initialize population.
ii) Selection according fitness.
iii) Crossover between selected chromosomes.
iv) Perform mutation.
v) Repeat cycle till true condition is true.



* A* Search Algorithm:

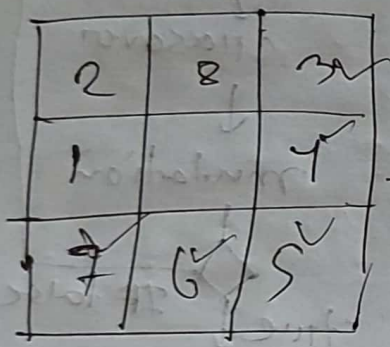




$f(n) = g(n) + h(n)$

$g(n)$ ↓ depth of node

$h(n)$ ↓ number of misplaced tiles

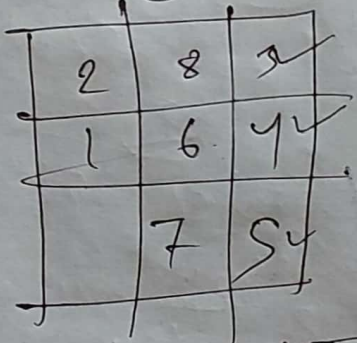


$g(n) = 0$

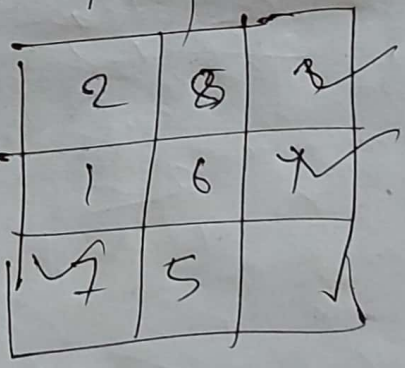
$h(n) = 4$

$f(n) = 4$

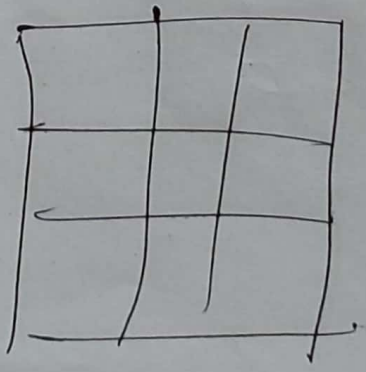
$3+1 = 4$

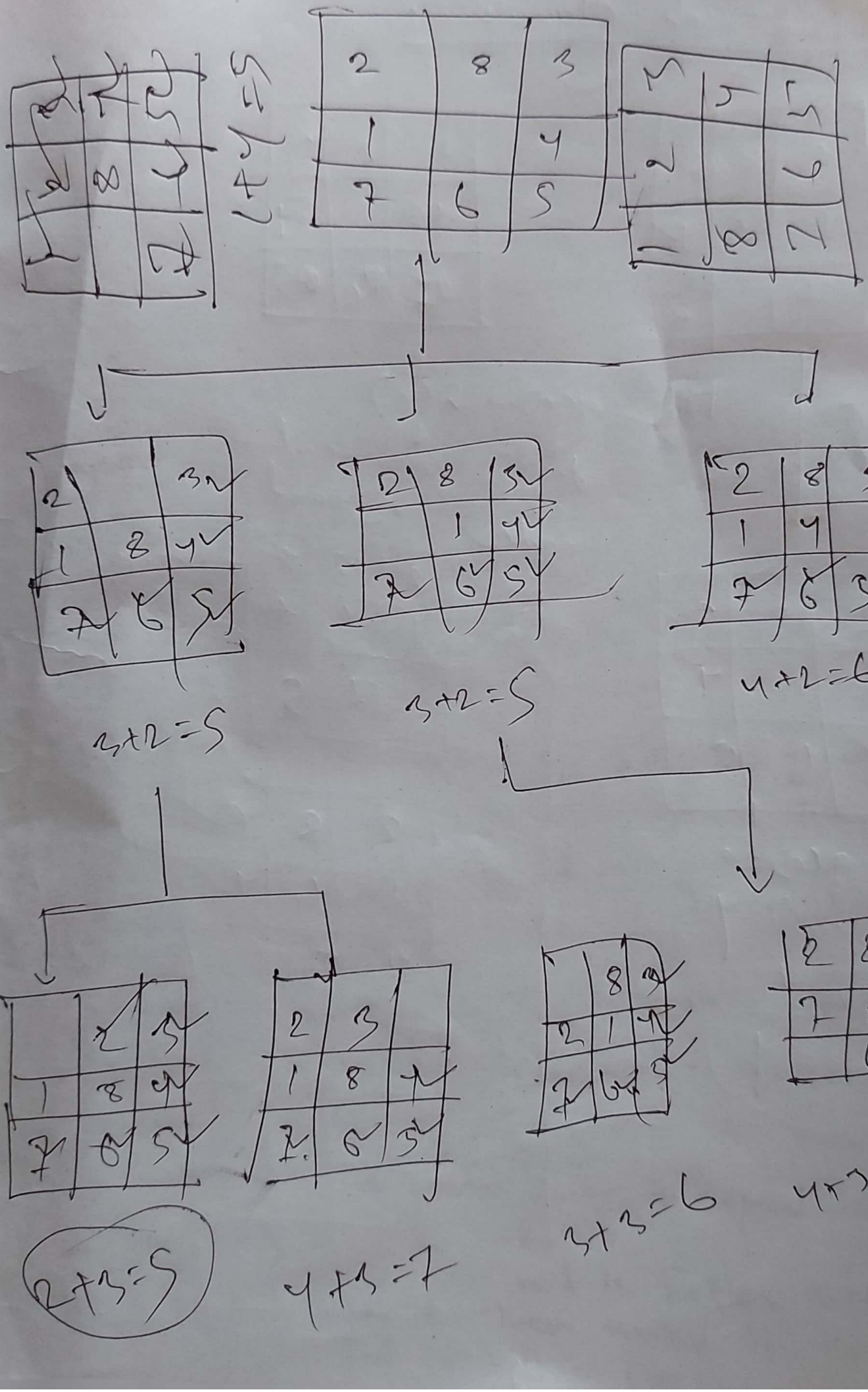


$5+1 = 6$



$5+1 = 6$




4+1=5
3+2=5
4+2=6
3+2=5
2+3=5
4+3=7
3+3=6
4+3=7

Algorithm of Simulated Annealing:

(i) Evaluate the initial state, if it also goal state then return it and stop.
otherwise continue with the initial state as current state.

(ii) Initialize Best_so_far = current state.

(iii) Initialize T according to the annealing Schedule

(iv) Loop until a solution is found or there are no ~~current~~ new operators left to be applied is the current state.

a) Select an operator that has not yet been applied to the current state and apply it to produce new state.

b) Evaluate the new state,

$$\Delta E = (\text{value of current}) - (\text{value of new state})$$

→ If new state is goal state, then return and stop

→ If not but is better than the current state then make it current state. set it best_so_far.

→ If it is not better than current state then make the current state with probability p'

(iv) Revise T as necessary according to the annealing schedule.

(v) Return best-so-far as the answer